Arvustuste lisamine ja kustutamine

11IT · Veebirakendused · 9. tund · konspekt

Ka siin selgitan kõik mõisted uuesti, kordamine on meelega. Koodi käivitamiseks on vaja Node.js-i, npm-i, VS Code'i ja söökla projekti, mille saad ka kursuse lehelt alla laadida. Lahenda „Proovi ise" kastid, vastused on lõpus. Veebilehel on **Päringulabor**, serveri simulatsioon, mis töötab brauseris ja kus saad päringuid proovida ilma midagi paigaldamata.

Pärast seda tundi oskad:
• valida õige meetodi: GET, POST või DELETE;
• kirjutada serverile marsruudid, mis lisavad ja kustutavad;
• saata Reactist päringu koos andmetega ja lugeda staatusekoodi.

Kõige tähtsam on kontrollitud arvustuse lisamine. Kustutamine on järgmine etapp. Läbivalt võrdleme kõike restoraniga.

Probleem

Praegu saab meie rakendus andmeid ainult lugeda. Pärast lehe värskendamist kaovad brauseris lisatud arvustused. Serverist saadud algsed arvustused jäävad alles. Miks? Sest uus arvustus jõudis ainult sinu brauseri mällu ja server ei kuulnud sellest midagi.

Täna paneme lehe serverile ütlema: „võta see arvustus vastu" ja „kustuta see ära".

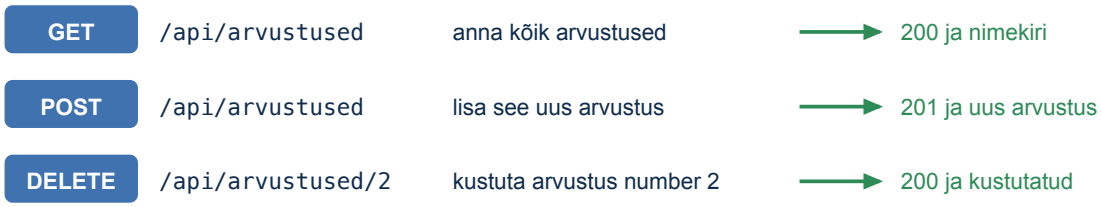
Kes on kes

Restoranis	Veebis	Mis see on
Sina laua taga	**klient**	Programm, mis saadab päringuid: brauser ja Reacti leht.
Tellimus	**päring**	Mida klient serverilt tahab.
Köök	**server**	Programm, mis ootab päringuid ja vastab. Meil Express.
See, mis lauale tuuakse	**vastus**	See, mille server tagasi saadab.

Meil töötab korraga kaks programmi, kumbki oma terminalis. **Vite** (`npm run dev`, tavaliselt port 5173) annab brauserile lehe. **Express** (`node index.js` kaustas `server`, port 3000) hoiab arvustusi. **Port** on number, mis ütleb, millisele programmile arvutis päring läheb. Leht küsib aadressilt `/api/...` ja Vite saadab päringu serverile edasi nagu teenindaja, kes viib tellimuse kööki.

Meetod: mida sa tahad teha?

Restoranis võid menüüd vaadata, tellida või tellimuse tühistada. Päringul on samuti lisaks aadressile **meetod**, mis ütleb, mida teha. Sama aadress, eri meetod: eri tegevus.



Sinine on meetod, siis aadress, tähendus ja mida server vastab.

Meetod	Tähendus	Restoranis
`GET`	loe, ära muuda midagi	Vaatad menüüd
`POST`	lisa midagi uut	Annad tellimuse
`DELETE`	kustuta	Tühistad tellimuse

Aadressiribalt saad selles näites teha `GET`-päringu. `POST`-päringu proovimiseks kasutame `fetch()`-i või Päringulaborit.

> **Proovi ise 1.** Mis meetodit kasutaksid: (a) tahad näha kõiki arvustusi, (b) tahad arvustuse number 3 ära võtta, (c) tahad uue arvustuse lisada?

Staatusekood: kuidas läks?

Server vastab alati ka kolmekohalise numbriga, **staatusekoodiga**. Sa oled ühte juba näinud: 404 ehk „ei leitud“. Esimene number näitab vastuse üldist liiki:

Kood	Tähendus	Millal meie server seda kasutab
200	OK, kõik läks hästi	Arvustused anti välja või arvustus kustutati
201	Loodud	Uus arvustus lisati
400	Halb päring: päringus on midagi valesti	Toit on puudu või hinne on 9
404	Ei leitud	Sellise numbriga arvustust pole
500	Serveri ootamatu viga	Seda ei tee me meelega

Rusikareegel: 2-ga algavad tähendavad õnnestumist, 4-ga algavad probleemi päringuga ja 5-ga algavad ootamatut viga serveris.

> **Proovi ise 2.** Saadad arvustuse, mille hinne on 9. Millise staatusekoodiga peaks server vastama? Miks?

Serveri pool: express.json ja req.body

Serveri failis `server/index.js` on **marsruudid**. Marsruut on reegel: „kui tuleb selline päring, tee nii“. Seni oli meil ainult `app.get`. Lisamiseks on `app.post`:

```
app.post('/api/arvustused', (req, res) => {
  // siia tuleb kood
});
```

Funktsioon saab kaks asja. `req` (ingl *request*) on päring: kõik, mida klient saatis. `res` (*response*) on vastus: selle kaudu saadame midagi tagasi. Märk `=>` teeb **noolefunktsiooni**, see on lühidalt kirjutatud funktsioon.

Klient saadab uue arvustuse **päringu kehas** (ingl *body*), meil JSON-tekstina. Aadress ja meetod ütlevad, mida teha, keha on tellimuse sisu. JSON-keha parsimiseks ehk JavaScripti väärtuseks lugemiseks on see rida, mis peab olema pärast `app`-i loomist ja enne marsruute:

```
app.use(express.json());
```

Nüüd saad andmeid lugeda kujul `req.body`. Kui JSON-keha pole parsitud, võib `req.body` olla `undefined` ehk väärtus puudub. Meie praegune kood annaks selle lahtivõtmisel vea.

```
const { toit, hinne, kommentaar } = req.body;
```

Seda kirjutusviisi nimetatakse lahtivõtmiseks. See on lihtsalt lühem kuju kolmest reast: `const toit = req.body.toit;` ja sama teistega.

Ära usalda kedagi: kontrolli andmeid

Vormis on hinne valikmenüüs 1 kuni 5, kas pole siis kõik korras? Ei ole. Päringut saab saata ka ilma sinu vormita ja brauseris saab koodi muuta. Seepärast peab server ise kontrollima.

Mõned märgid enne koodi: `===` tähendab „on täpselt võrdne“, `!==` „ei ole võrdne“, `&&` „ja“, `||` „või“ ning `!` „ei“. `true` on tõene ja `false` väär.

```
  const toitOnKorras = typeof toit === 'string' && toit.trim() !== '';
  const kommentaarOnKorras = typeof kommentaar === 'string' && kommentaar.trim() !== '';
  const hinneOnKorras = Number.isInteger(hinne) && hinne >= 1 && hinne <= 5;

  if (!toitOnKorras || !kommentaarOnKorras || !hinneOnKorras) {
    res.status(400).json({ viga: 'Täida kõik lahtrid, hinne on 1 kuni 5.' });
    return;
  }
```

Kood	Mida see teeb
`typeof toit === 'string'`	toit peab olema tekst.
`toit.trim() !== ''`	trim() eemaldab tühikud algusest ja lõpust. Alles peab jääma midagi.
`Number.isInteger(hinne)`	hinne peab olema täisarv. Tekst "5" ei sobi.
`hinne >= 1 && hinne <= 5`	hinne on lubatud vahemikus.
`if (!toitOnKorras || ...`	Kui vähemalt üks tingimus pole täidetud.
`res.status(400)`	Määrab staatusekoodi.
`.json({ viga: '...' })`	Saadab veateate JSON-ina.
`return;`	Lõpetab funktsiooni kohe, nii et arvustust ei lisata. Ilma selleta läheks kood edasi.

Uue arvustuse tegemine

```
  const uus = { id: jargmineId, toit: toit.trim(), hinne, kommentaar: kommentaar.trim() };
  jargmineId = jargmineId + 1;
  arvustused = [uus, ...arvustused];
  res.status(201).json(uus);
```

Kood	Mida see teeb
`id: jargmineId`	Igal arvustusel peab olema oma number. Numbri paneb server.
`toit: toit.trim()`	Salvestame teksti ilma üleliigsete tühikuteta.
`jargmineId = jargmineId + 1;`	Järgmine saab ühe võrra suurema numbri.
`[uus, ...arvustused]`	Uus massiiv: ees on uus. Kolm punkti koodis laotavad vanad elemendid selle järele.
`res.status(201).json(uus);`	Vastab koodiga 201 ja saadab uue arvustuse koos numbriga tagasi.

Selleks, et muutujatele `arvustused` ja `jargmineId` saaks anda uue väärtuse, peavad need olema kirjutatud sõnaga `let`, mitte `const`.

Kustutamine ja aadressi sees olev number

Kuidas öelda, millist arvustust kustutada? Number pannakse aadressi: `/api/arvustused/2`. Marsruudis tähistab seda koht, mis algab kooloniga. `.find()` otsib massiivist esimese sobiva elemendi. `.filter()` teeb uue massiivi nendest, mis tingimusele vastavad.

```
app.delete('/api/arvustused/:id', (req, res) => {
  const id = Number(req.params.id);
  const leitud = arvustused.find((a) => a.id === id);

  if (!leitud) {
    res.status(404).json({ viga: 'Sellist arvustust pole.' });
    return;
  }

  arvustused = arvustused.filter((a) => a.id !== id);
  res.json(leitud);
});
```

Kood	Mida see teeb
`'/api/arvustused/:id'`	:id tähendab „siia võib tulla mis tahes väärtus, nimetame seda id“.
`req.params.id`	See väärtus. Aadressil /api/arvustused/2 on see tekst "2", sest aadress on alati tekst.
`Number(...)`	Teeb tekstist "2" arvu 2.
`.find((a) => a.id === id)`	Kui ei leia, annab undefined.
`if (!leitud) {`	Kui sellist polnud, vasta 404.
`.filter((a) => a.id !== id)`	Jätab alles kõik, mille id ei ole see. Nii jääb kustutatav välja.

Proovi ise 3. Massiivis on id-d 1, 2 ja 3. Saabub `DELETE /api/arvustused/7`. Millise staatusekoodi saad? Mis juhtub massiiviga?

Kliendi pool: fetch koos meetodiga

Seni kirjutasime `fetch('/api/arvustused')` ja see tegi `GET`-päringu. Teiste meetodite puhul on vaja teist argumenti: objekti, kus on päringu seaded.

```
const vastus = await fetch('/api/arvustused', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ toit, hinne, kommentaar }),
});
const andmed = await vastus.json();
```

Kood	Mida see teeb
`await`	Kasuta tulemust, kui see on kohal. Funktsiooni ette peab siis kirjutama async.
`method: 'POST',`	Mis meetodiga saata.
`headers: { 'Content-Type': 'application/json' },`	Päringu päised on lisainfo päringu kohta. See ütleb serverile: keha on JSON. Ilma selleta ei parsi express.json() keha.
`JSON.stringify({ ... })`	Teeb objektist JSON-teksti, sest meie server ootab keha JSON-vormingus.
`vastus.json()`	Loeb serveri JSON-vastuse JavaScripti väärtuseks.

Kontrollime ka tulemust. `vastus.ok` on `true`, kui staatus on 200–299. Kui ei, näitame serveri veateadet, mis on kohas `andmed.viga`:

```
if (!vastus.ok) {
  setViga(andmed.viga);
  return;
}
setArvustused((praegused) => [andmed, ...praegused]);
```

`useState` on Reacti mälu ehk olek. Nüüd paneme nimekirja serveri vastuse, sest alles selles on õige `id`. Pane tähele, et `setArvustused` saab nüüd funktsiooni. `praegused` on oleku kõige värskem väärtus. See on vajalik, sest vastus võib tulla alles hiljem: kui vahepeal jõudis mõni teine päring nimekirja muuta, ei kirjutata seda muudatust üle.

Server võib ka üldse mitte vastata. Selleks on kogu päring pandud `try`-plokki, nagu eelmises tunnis.

Kustutamisel pole keha vaja, sest kõik on aadressis:

```
const vastus = await fetch(`/api/arvustused/${id}`, { method: 'DELETE' });
```

Tagurpidi ülakomad `` ` `` teevad **malliteksti**: `${id}` asemele pannakse muutuja väärtus.

Kuidas nupp saab kustutada?

Kustuta-nupp on komponendis `ArvustuseKaart`, aga nimekiri elab komponendis `App`. Lahendus: `App` annab kaardile funktsiooni `kustuta` **propsina**. Prop on väärtus, mille komponent väljast saab, nagu funktsiooni argument. Kaart ütleb nupuvajutusel: „kutsu see välja minu numbriga“.

```
<button type="button" onClick={() => kustuta(arvustus.id)}>Kustuta</button>
```

Miks `() => kustuta(arvustus.id)`, mitte lihtsalt `kustuta(arvustus.id)`? Teisel juhul kutsutakse funktsioon välja juba joonistamise ajal, ilma nupuvajutuseta, ja see võib arvustusi kustutada. TypeScript võib selle vea juba enne käivitamist märgata.

Aga miks server unustab?

Pane server `Ctrl+C`-ga kinni ja käivita uuesti. Sinu lisatud arvustused on kadunud ja kustutatud on tagasi. Miks? Massiiv `arvustused` elab programmi **mälus**, nagu tekst tahvlil. Programm kinni, tahvel puhas. Käivitamisel kirjutatakse failist uuesti need kolm esimest.

Võrreldes varasemaga on palju paremaks läinud: arvustused jäävad alles lehe värskendamisel. Ühiseid arvustusi näevad kliendid, mis kasutavad sama serverit. Iga õpilase enda arvutis töötaval serveril on eraldi andmed.

Andmete säilimiseks pärast serveri sulgemist tuleb need püsivalt salvestada, näiteks faili või andmebaasi. Edaspidi kasutame selleks andmebaasi.

Kui midagi ei tööta

Mida näed	Võimalik põhjus	Mida kontrollida
Server ei reageeri koodimuudatusele	Serverit pole pärast koodi muutmist uuesti käivitatud.	`Ctrl+C` ja `node index.js`.
Staatus 404, `Cannot POST`	Sellise meetodi ja aadressiga marsruuti pole.	Aadress ja meetod täht-tähelt.
Staatus 500, `Cannot destructure`	`req.body` on `undefined`.	Kas `app.use(express.json());` on enne marsruute ja kas fetch-is on `headers`.
Staatus 400, kuigi kõik on täidetud	Hinne läks tekstina või lahter on ainult tühikuid täis.	Vormi `Number(e.target.value)` ja F12 all Network → Payload.
Arvustused kaovad ilma nupuvajutuseta	`onClick={kustuta(arvustus.id)}`	Kas ees on `() =>`.
„Serveriga ei saanud ühendust“	Server ei tööta või vastus pole JSON.	Serveri terminal ja Network.

Sõnastik

Sõna	Tähendus
meetod	mida päring teha tahab: GET, POST, DELETE
staatusekood	kolmekohaline number vastuses, mis näitab vastuse liiki
marsruut	serveri reegel: selle meetodi ja aadressi peale tee nii
päringu keha (body)	päringuga kaasa saadetud andmed
päringu päis (header)	lisainfo päringu kohta, näiteks mis kujul keha on
parsimine	teksti lugemine andmeteks
req, res	päring ja vastus serveri marsruudis
req.params	aadressi muutuvad osad, näiteks :id
JSON.stringify	teeb objektist JSON-teksti
prop	väärtus, mille komponent väljast saab
andmebaas	programm, mis hoiab andmeid püsivalt, nii et need jäävad alles

„Proovi ise“ vastused

1. (a) GET, (b) DELETE, (c) POST.
2. 400, sest päringus olev hinne ei ole lubatud.
3. 404. Massiiv ei muutu, sest `return` lõpetab funktsiooni enne `filter`-it.